ECSE 551 Assignment 1

Tingyi Chen, Raisa Chowdhury, John Wu

***Abstract*—This report implements and evaluates machine learning models from scratch using NumPy. For regression on the Diabetes dataset (411 patients after outlier removal, 10 features), ElasticNet and Lasso models with gradient descent and soft thresholding were implemented and compared against sklearn's `LinearRegression`. For classification on the Digits dataset (1797 images, 64 pixels), Multiclass SVM using a one-vs-rest strategy and Multilayer Perceptron were implemented and compared against `RandomForestClassifier`. Hyperparameter tuning optimized learning rates, regularization parameters, hidden layer sizes, and iteration counts. Performance evaluation used MSE for regression and accuracy for classification. The Lasso and ElasticNet models achieved a test MSE of approximately 2840, comparable to sklearn's `LinearRegression`. The Multiclass SVM achieved 95.56% test accuracy with optimal regularisation ($\lambda = 0.01$). Results demonstrate that scratch implementations achieve competitive performance relative to sklearn baselines.**

I. Introduction

Machine learning algorithms play a critical role in predictive analytics, particularly in healthcare applications where predictions lead to clinical decision-making. This work implements core supervised learning algorithms from scratch to understand their mathematical foundations. The objectives are to develop regression and classification models using only NumPy [1], then compare performance against scikit-learn baselines and analyse hyperparameter effects on model behaviour.

Two datasets were selected for analysis from the scikit-learn library [2]. The Diabetes dataset contains 442 patients with 10 baseline features (age, sex, BMI, blood pressure, serum measurements) predicting disease progression one year after baseline. After IQR-based outlier removal, 411 samples were used for regression modelling. The Digits dataset comprises 1797 handwritten digit images (0–9) represented as 8×8 pixel arrays (64 features) for classification tasks.

For regression, ElasticNet and Lasso models with gradient descent and soft thresholding were implemented from scratch, combining L1 and L2 regularisation for feature selection and weight control. Performance was compared against `LinearRegression` from scikit-learn. For classification, Multiclass SVM using a one-vs-rest strategy with hinge loss and Multilayer Perceptron with ReLU activation and backpropagation were implemented from scratch and compared against `RandomForestClassifier`. All implementations used only NumPy, requiring manual gradient computation and parameter updates.

Hyperparameter tuning optimized learning rates (η), regularization penalties (λ, λ_1, λ_2), iteration counts, and hidden layer sizes. Models were evaluated using mean squared error for regression and accuracy for classification, with results showing that scratch implementations achieve competitive performance when properly tuned.

II. Methodology

A. Lasso Regression

Lasso Regression is a linear regression model with L1 regularization [3]. Consider a linear regression model without any regularization. The goal of this simple linear regression model is to find the $\hat{y}_i = W^T x + b$, where W is the weight vector and b is the bias, that minimizes its loss function

$$\mathcal{L}(W, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y})^2 \tag{1}$$

This is the mean squared error (MSE), representing the distance between the predicted linear function and the sample data points.

Lasso Regression adds a L1 regularization term in addition to this loss function of the simple linear regression, therefore Lasso Regression targets to minimize

$$\mathcal{L}(W, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y})^2 + \lambda \sum_{j=1}^{m} |w_j| \tag{2}$$

λ is the L1 penalty parameter and w_j is the weight for feature j. The addition of the L1 regularization term to the loss function encourages sparsity; it allows weights for relatively irrelevant features to become 0, therefore performing automatic feature selection.

The gradients of Lasso Regression are therefore

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{2}{n} \sum_{i=1}^{n} (\hat{y} - y_i) x_{ij} + \lambda sign(w_j) \tag{3}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n} \sum_{i=1}^{n} (\hat{y} - y_i) \tag{4}$$

However, $sign(w_j)$ is undefined when $w_j = 0$, therefore soft thresholding is used to handle the update of the L1 regularization term. The formula for soft thresholding is the following:

$$S(w_j) = sign(w_j) max(|w_j| - \eta\lambda, 0) \tag{5}$$

or in the piecewise form,

$$S(w_j) = \begin{cases} w_j - \eta\lambda, & \text{if } w_j > \eta\lambda \\ 0, & \text{if } |w_j| \leq \eta\lambda \\ w_j + \eta\lambda, & \text{if } w_j < -\eta\lambda \end{cases} \tag{6}$$

Soft thresholding shrinks all weights toward 0, and when a certain weight is very close to 0, it makes it 0.

Updating of the model is done repeatedly by gradient descent, where

$$w_j(new) = w_j(old) - \frac{\partial \mathcal{L}}{\partial w_j} \tag{7}$$

$$b(new) = b(old) - \frac{\partial \mathcal{L}}{\partial b} \tag{8}$$

B. ElasticNet Regression

ElasticNet Regression is another linear regression model that adds an L2 regularization term [4] on top of Lasso Regression. Its goal is to minimize the loss function

$$\mathcal{L}(W, b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y})^2 + \lambda_1 \sum_{j=1}^{m}|w_j| + \lambda_2 \sum_{j=1}^{m} w_j^2 \tag{9}$$

The L2 regularization term shrinks all weights toward 0 much quicker than the L1 regularization term, but never sets any weight to 0 no matter how strong the penalty is. Therefore, by combining L1 and L2 regularization, ElasticNet Regression enforces exact zeros for irrelevant weights with the L1 regularization term and shrinks all weights toward 0 quickly using the L2 regularization term.

It is also worth noticing that when $\lambda_1 = 0$, ElasticNet reduces to Ridge Regression, while when $\lambda_2 = 0$, ElasticNet reduces to Lasso Regression.

The gradients of ElasticNet Regression are the following:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y} - y_i)x_{ij} + \lambda_1 sign(w_j) + 2\lambda_2 w_j \tag{10}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y} - y_i) \tag{11}$$

Due to the same reason discussed in the section above, soft thresholding is used to handle the gradient of the L1 regularization term. Updating of the model is done repeatedly by gradient descent.

C. Multiclass SVM

Dataset and Train/Test Split:

The dataset, loaded using scikit-learn Digits dataset (containing 1,797 samples of 8x8 grayscale images of handwritten digits), was divided into 70% training and 30% testing using a fixed random seed with state 42.

Model: One-vs-Rest Multiclass SVM A Support Vector Machine (SVM) [5] finds a hyperplane that optimally separates classes in feature space.

Since the Digits dataset has 10 classes, a One-vs-Rest (OvR) strategy is used: one binary SVM is trained per class c, where labels are set to $+1$ if the sample belongs to class c and -1 otherwise. For binary classification of each class, the decision boundary is defined as:

$$f(x) = w^\top x + b \tag{12}$$

where $w \in \mathbb{R}^d$ is the weight vector and b is the bias. A sample is classified as positive if $f(x) \geq 1$ and negative if $f(x) \leq -1$.

At prediction time, the class with the highest decision score is selected:

$$\hat{y} = \arg\max_c \ (w_c^\top x + b_c) \tag{13}$$

The SVM objective minimizes the regularized hinge loss:

$$\mathcal{L}(w) = \lambda \|w\|^2 + \frac{1}{n}\sum_{i=1}^{n}\max(0, 1 - y_i(w^\top x_i + b)) \tag{14}$$

The gradient used for the subgradient descent update is:

$$\frac{\partial \mathcal{L}}{\partial w} = \begin{cases} 2\lambda w & \text{if } y_i(w^\top x_i + b) \geq 1 \\ 2\lambda w - y_i x_i & \text{otherwise} \end{cases} \tag{15}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \begin{cases} 0 & \text{if } y_i(w^\top x_i + b) \geq 1 \\ -y_i & \text{otherwise} \end{cases} \tag{16}$$

Weights and biases are updated via gradient descent: $w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$, where η is the learning rate.

The term $\lambda\|w\|^2$ is an L2 regularization penalty (also called Ridge regularization) that discourages large weights, reducing overfitting by penalizing model complexity. The hyperparameter λ controls the strength of this penalty.

D. MLP

Multilayer Perceptron (MLP) is a neural network that consists of an input layer, one or more hidden layers, and an output layer [6]. For the purpose of this assignment, MLP is assumed to do classification and have only one hidden layer. It is used to find the complex non-linear relationship between the input features and the outputs.

With the example shown in figure 1, every neuron in the input layer is fully connected to every neuron in the hidden layer, and every neuron in the hidden layer is fully connected to every neuron in the output layer.

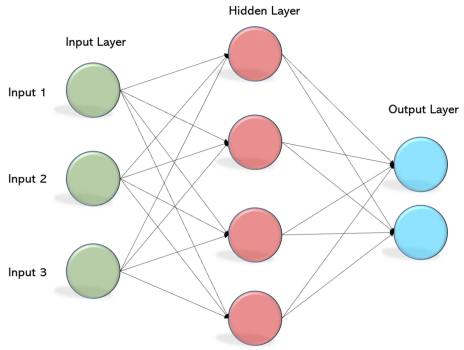

Fig. 1: MLP example with three input neurons, four hidden layer neurons, and two outputs.

Each connection between two neurons has a weight. All neurons except for the input ones have their own biases. The hidden layer neurons are calculated by taking the weighted sum of all input neurons, adding their own biases, and then applying an activation function:

$$A_j = ReLU(\sum_i x_i w_{ij} + b_j) \tag{17}$$

A_j is hidden neuron j, x_i is input neuron i, w_{ij} is the weight from input neuron i to hidden neuron j, and b_j is the bias for hidden neuron j.

The activation function here is $ReLU(z)$, which is a simple function that introduces non-linearity to MLP, defined as

$$ReLU(z) = max(0, z) \tag{18}$$

The output layer neurons are calculated by again, taking the weighted sum of all hidden layer neurons, adding their own biases, and then applying a softmax function:

$$\hat{y}_k = softmax(\sum_j A_j w_{jk} + b_k) \tag{19}$$

For the purpose of classification, softmax is used to turn the set of output neurons into probabilities, defined as

$$softmax(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \tag{20}$$

$\hat{y}_k$ is output neuron k, w_{jk} is the weight from hidden neuron j to output neuron k, and b_k is the bias for output neuron k.

Together, this is called the forward propagation of MLP that links input neurons and output neurons.

Backpropagation, on the other hand, is the key that updates the weights and biases to ensure the correctness of the MLP model. Assume that the true labels y are one-hot encoded, then the cross entropy loss function of MLP is defined as

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{C} y_{ik} log(\hat{y}_k) \tag{21}$$

By summing all the true labels multiplied by their corresponding probabilities, this loss function penalizes the model when it gives low probabilities to the correct classes.

With the definition of matrices $Z_{hidden} = XW_{input\_hidden} + B_{hidden}$ being hidden neuron matrix before $ReLU$ and $Z_{output} = AW_{hidden\_output} + B_{output}$ being output neuron matrix before $softmax$, the output error is the following matrix

$$\delta_{output} = \frac{\partial \mathcal{L}}{\partial Z_{output}} = \hat{Y} - Y \tag{22}$$

Then the hidden layer error, by chain rule, is the following matrix

$$\begin{aligned} \delta_{hidden} &= \frac{\partial \mathcal{L}}{\partial Z_{hidden}} \\ &= \frac{\partial \mathcal{L}}{\partial Z_{output}} \frac{\partial Z_{output}}{\partial A} \frac{\partial A}{\partial Z_{hidden}} \\ &= \delta_{output} W^T_{hidden_output} \cdot ReLU'(Z_{hidden}) \end{aligned} \tag{23}$$

The gradients are calculated as follows

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_{hidden\_output}} &= \frac{1}{n} \frac{\partial \mathcal{L}}{\partial Z_{output}} \frac{\partial Z_{output}}{\partial W_{hidden\_output}} \\ &= \frac{1}{n} A^T \delta_{output} \end{aligned} \tag{24}$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial B_{\text{output}}} &= \frac{1}{n} \sum_{i=1}^{n} \frac{\partial \mathcal{L}^{(i)}}{\partial Z^{(i)}_{\text{output}}} \frac{\partial Z^{(i)}_{\text{output}}}{\partial B_{\text{output}}} \\ &= \frac{1}{n} \sum_{i=1}^{n} \delta^{(i)}_{\text{output}} \end{aligned} \tag{25}$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_{input\_hidden}} &= \frac{1}{n} \frac{\partial \mathcal{L}}{\partial Z_{hidden}} \frac{\partial Z_{hidden}}{\partial W_{input\_hidden}} \\ &= \frac{1}{n} X^T \delta_{hidden} \end{aligned} \tag{26}$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial B_{\text{hidden}}} &= \frac{1}{n} \sum_{i=1}^{n} \frac{\partial \mathcal{L}^{(i)}}{\partial Z^{(i)}_{\text{hidden}}} \frac{\partial Z^{(i)}_{\text{hidden}}}{\partial B_{\text{hidden}}} \\ &= \frac{1}{n} \sum_{i=1}^{n} \delta^{(i)}_{\text{hidden}} \end{aligned} \tag{27}$$

Updating of the MLP model is done repeatedly by gradient descent.

III. Datasets

A. Diabetes Dataset

The dataset contains 442 patients with 10 standardized features (age, sex, bmi, bp, s1-s6) and a continuous target (disease progression). After IQR-based outlier removal [7], 411 samples remained. Features have a mean ≈ 0 and std ≈ 0.048, indicating pre-standardisation. Target has mean = 152.13, std = 77.09, range [25, 346].

Correlation analysis identified BMI and s5 as the strongest predictors of disease progression. Scatter plots confirmed positive linear relationships between these features and the target. Box plots showed symmetric distributions with outliers successfully removed.

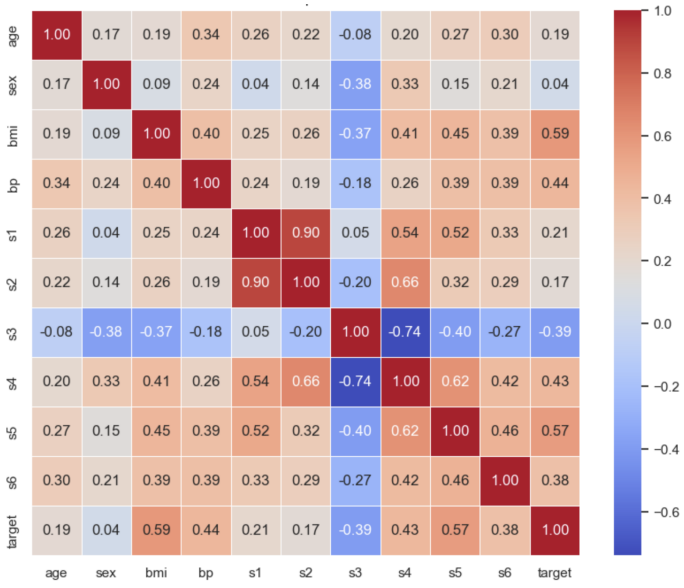

Fig. 2: Correlation Heatmap of Diabetes Dataset Features

B. Digits Dataset

The dataset contains 1797 grayscale images of handwritten digits (0–9), each represented as 64 pixel intensities (0 = white, 16 = black). There were no missing values detected.

Pixel correlation analysis showed edge pixels have weak correlations with digit class, while central pixels are more discriminative. Histograms of selected pixels revealed that edge pixels are predominantly zero (white), while central pixels show multimodal distributions reflecting diverse digit patterns.

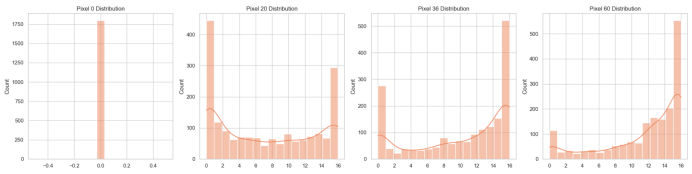

Fig. 3: Distribution of selected pixels

IV. Results

A. Lasso and ElasticNet Regression

Before training the regression models, the diabetes dataset is standardized such that the L1 penalty term in the loss functions would not favor larger-scaled features.

The implemented Lasso Regression model has hyperparameters including the L1 regularization penalty λ, the learning rate η, and the number of iterations used during optimization.

The implemented ElasticNet Regression model has hyperparameters including the L1 and L2 regularization penalties λ_1 and λ_2, the learning rate η, and the number of iterations used during optimization.

With L1 and L2 penalties $\lambda = 0.1$, $\eta = 0.01$, and number of iterations of 2000, the performance of the implemented Lasso and ElasticNet Regression models and the models from sklearn are compared in figure 4.

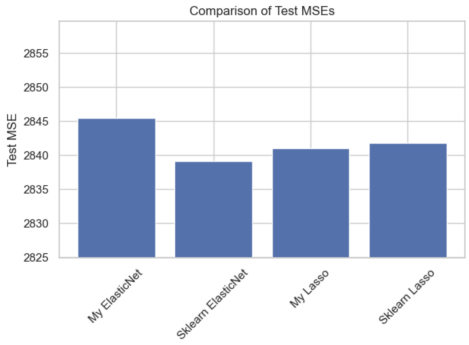

Fig. 4: Test MSE of Lasso and ElasticNet Regressions.

All four test MSEs are around 2840; the small differences between them prove the correctness of the implemented models. It can be said that the implemented models have very similar performance with the same hyperparameters.

For both Lasso and ElasticNet Regression, it is intuitive that increasing the number of optimization iterations will reduce the test and training MSE, but there is an upper limit: once the model has converged, more iterations will not help. This is shown in figure 5 using Lasso Regression as an example. There is a significant decrease in the test MSE when increasing the number of iterations from 100 to 200, which is expected. Following that, there is a slight increase in MSE since the model is still not fully converged. After that, even if the number of iterations is 100 times larger, there is no significant change in test MSE. Modifying the number of iterations has the same effect on ElasticNet Regression.

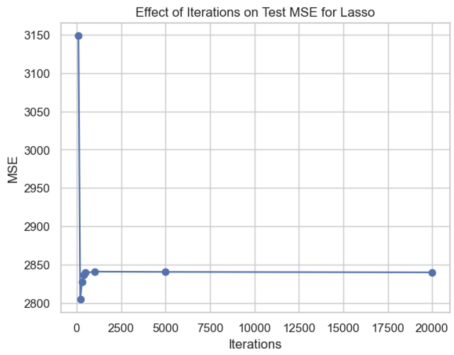

Fig. 5: Effect of Iterations on test MSE for Lasso Regression, λ = 0.1, η = 0.01.

For the learning rate η, a value that is too small will make model fitting take too many iterations and therefore be inefficient, but a value that is too large will sometimes make the gradient steps too big, causing undesirable behavior such as oscillation. Varying η from 0.0001 to 0.1 for the implemented Lasso and ElasticNet models results in very close test MSEs as long as there are enough optimization iterations.

The most important hyperparameters are the L1 and L2 penalties for Lasso and ElasticNet Regression. For Lasso Regression, L1 penalty has a significant impact on the model, reflected through changes in MSEs and changes in learned coefficients (weights). If λ is set to be too small, the Lasso Regression will behave very similarly to the simple linear regression without any regularizations. If λ is set to be too large, most of the features will be filtered out, where their weights become 0, and therefore make the model too simple [8].

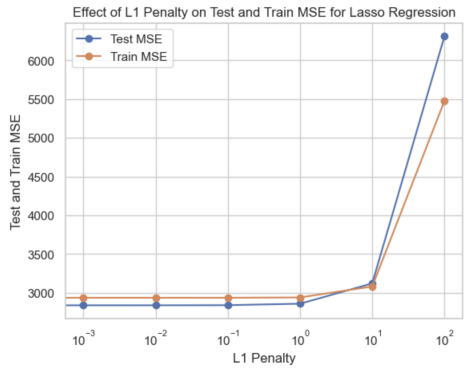

Fig. 6: Effect of L1 Penalty on test MSE for Lasso Regression, iterations = 2000, η = 0.01.

Shown in figure 6, the gap between train MSE and test MSE is small, which is a good sign that the model is working as expected. The test MSE significantly increases when the L1 penalty is increased from 10 to 100, whereas the test MSEs for all other L1 penalties are about the same. Referring to table I, the weights slowly converge to 0 as the L1 penalty increases before 1, and the number of 0s in the weights increases as the L1 penalty increases after 1. This is coherent with the theory that a large L1 penalty enforces feature selection. Those features that still have their weights remaining are the highly correlated features to diabetes disease progression, such as bmi and bp.

L1 λ	**Weights**
0.001	$-3.1, -9.2, 23.2, 14.1, -2.8, -4.3, -7.2, 3.9, 25.8, 2.0$
0.01	$-3.1, -9.2, 23.2, 14.1, -2.8, -4.2, -7.2, 3.9, 25.8, 2.0$
0.1	$-3.1, -9.1, 23.1, 14.0, -2.6, -4.2, -7.4, 3.6, 25.8, 2.0$
1	$-2.4, -8.2, 22.9, 13.4, -0.7, -3.4, -9.2, 0.4, 25.5, 1.5$
10	$-0.0, -0.5, 21.9, 8.6, -0.0, -0.0, -3.7, 0.0, 23.0, 0.0$
100	$0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.0, 0.0, 0.0, 0.0$

TABLE I: Effect of different L1 penalties on Lasso Regression weights and test MSE.

The ElasticNet Regression has another L2 penalty term on top of Lasso Regression. The L2 penalty focuses on quickly converging the weights toward 0, while the L1 penalty focuses on eliminating irrelevant features. Fixing the L2 penalty and plotting the L1 penalty vs. test MSE, the result ends up highly similar to figure 6, for the same reason explained above.

Fixing the L1 penalty and plotting the L2 penalty vs. test MSE, the result is shown in fig. 7, where the test MSE increases as the L2 penalty becomes too large.

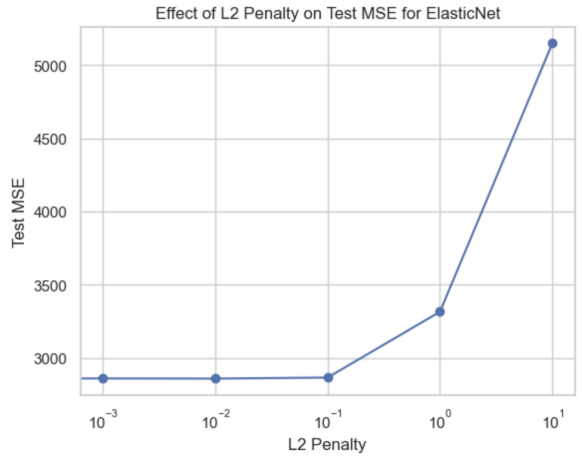

Fig. 7: Effect of L2 Penalty on test MSE for Lasso Regression, $\lambda_1 = 1$, iterations = 2000, η = 0.01.

This trend can be explained by referring to table II. As the L2 penalty increases, weights shrink to 0 a lot and therefore when the L2 penalty becomes too large, the weights cannot properly fit the data. Filtering of features are not obviously performed here since L1 penalty is set to 1 and the optimization iterations are not too big (2000).

Overall, Lasso Regression uses the L1 penalty to filter out less correlated features while slowly converging feature weights to 0. ElasticNet Regression uses both L1 and L2 penalties to not only filter out less correlated features, but also converge feature weights fast toward 0. Although they result in a generally similar test MSE on this diabetes dataset, they can be very useful in specialized scenarios such as datasets with a lot of irrelevant features.

L2 λ	**Weights**
0	$-2.4, -8.2, 22.9, 13.4, -0.7, -3.4, -9.2, 0.4, 25.5, 1.5$
0.001	$-2.4, -8.2, 22.8, 13.4, -0.7, -3.4, -9.2, 0.4, 25.5, 1.5$
0.01	$-2.3, -8.1, 22.7, 13.4, -0.5, -3.7, -9.1, 0.8, 25.1, 1.6$
0.1	$-1.6, -7.4, 21.3, 12.8, -0.0, -4.1, -8.1, 2.7, 22.3, 2.3$
1	$0.5, -3.2, 13.3, 8.9, 0.7, -0.9, -5.8, 4.3, 12.9, 3.9$
10	$0.8, -0.1, 3.1, 2.4, 0.9, 0.5, -1.8, 1.8, 3.1, 1.6$

TABLE II: Effect of different L2 penalties on ElasticNet Regression weights and test MSE.

B. Multiclass SVM

The model was evaluated across three values of $\lambda \in \{0.001, 0.01, 0.1\}$, with a fixed learning rate $\eta = 0.001$ and 1,000 iterations. The best test accuracy of 95.56% was achieved at $\lambda = 0.01$.

At $\lambda = 0.001$, the model achieves the highest training accuracy (97.77%) but a lower test accuracy (93.70%), indicating some overfitting; the weaker regularization allows the model to fit the training data more closely at the cost of generalization. At $\lambda = 0.01$, the gap between training (97.30%) and test (95.56%) accuracy narrows. At $\lambda = 0.1$, both training (93.87%) and test (93.33%) accuracy drop, indicating that the regularization penalty is too strong; the bigger penalty generalizes too heavily, making it difficult for the model to distinguish between classes. This can be seen in figure 8.

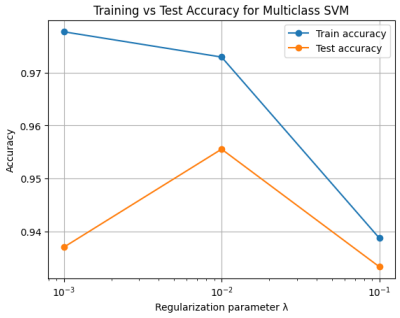

Fig. 8: Accuracy as a function of the regularization parameter λ.

As shown in figure 9, the confusion matrix at $\lambda = 0.01$ confirms strong per-class performance, with most predictions falling along the diagonal. The most notable off-diagonal entries include the model misclassifying the digit 3 as 8, the digit 9 as 8, and the digit 1 as 6 (very weak).

For learning rate tuning, three values $\eta \in \{0.0001, 0.001, 0.01\}$ were tested with $\lambda = 0.01$ fixed. At $\eta = 0.0001$, the model achieves a test accuracy of 95.37%, and at $\eta = 0.001$ it reaches the best performance of 95.56%. At $\eta = 0.01$, accuracy drops sharply to 90.56%, likely because the larger step size causes the gradient descent updates to overshoot, preventing convergence to a good

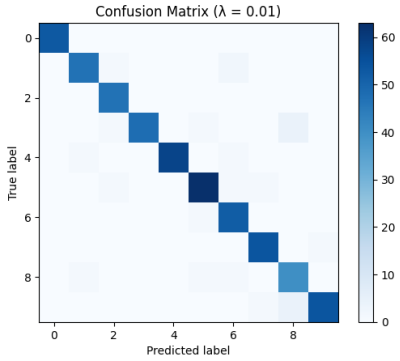

Fig. 9: Confusion matrix for the multiclass SVM at $\lambda = 0.01$.

solution. This confirms that $\eta = 0.001$ is the most suitable learning rate for this configuration. This can be seen below.

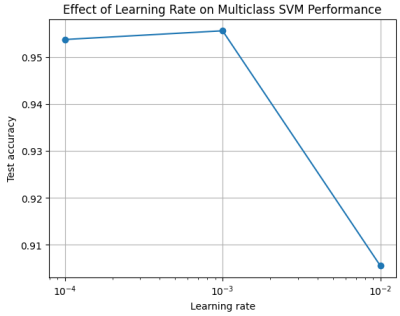

Fig. 10: Test accuracy as a function of learning rate η for the multiclass SVM (with $\lambda = 0.01$ fixed).

C. MLP

The MLP implemented to classify the digits has fixed 64 input neurons, each representing one pixel, and 10 output neurons, each representing a classified digit.

The hyperparameters for MLP include the learning rate η, the number of iterations used during optimization (epochs), and the number of hidden neurons.

Figure 11 shows the plot of epochs vs. test accuracy. As epochs increase, test accuracy converges to 1. This is an intuitive result since more epochs mean that the model is more optimized through a greater number of backpropagations.

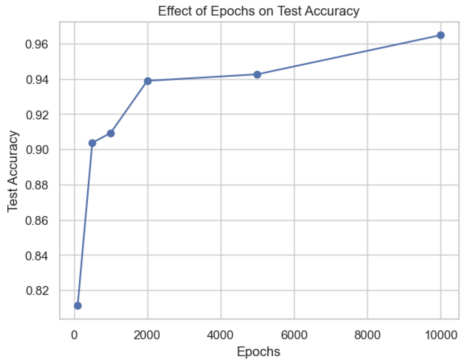

Fig. 11: Effect of Epochs on Test Accuracy MLP, hidden layer size = 32, η = 0.01.

Figure 12 shows the plot of the number of hidden layer neurons vs. test accuracy. It illustrates that increasing the hidden layer size generally increases test accuracy until a certain limit. More hidden neurons can capture more complex relationships between the input and output layers, but too many hidden neurons will lead to overfitting of the training data and eventually decrease the test accuracy. This penalty of too many hidden layer neurons does not show up in figure 12 because overfitting is less obvious on clean and small datasets.

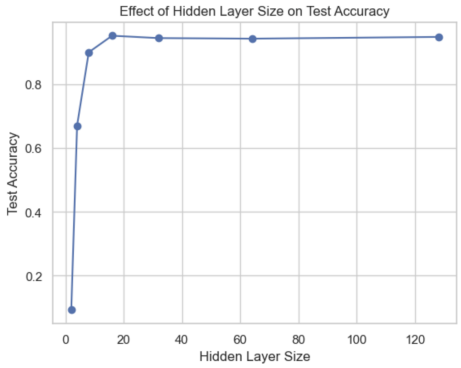

Fig. 12: Effect of Hidden Layer Size on Test Accuracy MLP, epochs = 2000, η = 0.01.

Figure 13 shows the plot of η vs. test accuracy. $\eta = 0.1$ has the best test accuracy since it converges the model faster, whereas smaller η values take more epochs to optimize fully and converge.

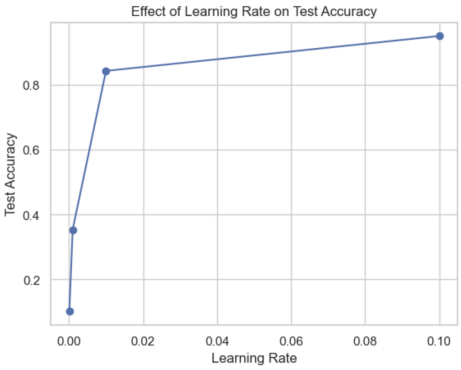

Fig. 13: Effect of Learning Rate on Test Accuracy MLP, epochs = 2000, hidden layer size = 32.

Random Forest Classifier (RFC) uses decision trees in parallel and takes the majority vote from them. It is a less expensive model compared with MLP, but it can miss deeply embedded relationships. With parameters epochs = 1000, η = 0.01, hidden layer size = 16 for MLP, and number of decision trees = 30 for RFC, RFC has a much cleaner confusion matrix than MLP (Figure 14). MLP, with a test accuracy of 0.87, confuses 5 with 9 or 8 with 2, whereas RFC has a test accuracy of 0.97, although neither of them are heavily trained. This demonstrates that for MLP to behave well even for simple datasets, it requires fine-tuning of the hyperparameters. RFC is a fast and precise classification model for simple datasets without much tuning [8].

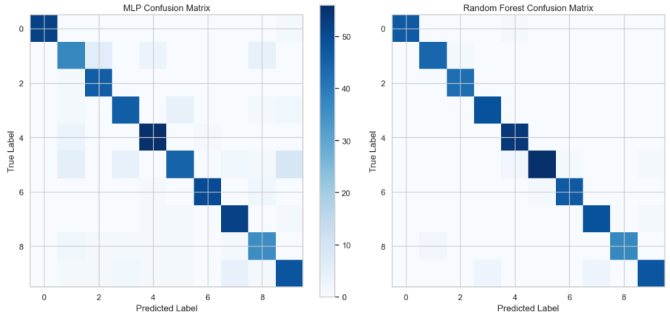

Fig. 14: Confusion Matrices of MLP and RFC.

Overall, with suitable hyperparameters, MLP can learn complex non-linear relationships between the inputs and outputs and make accurate classification predictions.

V. Discussion and Conclusion

This assignment implemented and evaluated four machine learning models, Lasso, ElasticNet, SVM, and MLP, from scratch. Results were validated against sklearn benchmarks, and hyperparameter experiments confirmed that all models are sensitive to regularization tuning, with too little leading to overfitting and too much degrading performance. Overall, the results aligned with the theoretical expectations of each model, signaling correct implementation.

Lasso and ElasticNet

The implemented models for Lasso and ElasticNet matched sklearn's performance, validating the implementations. The key finding is that the magnitude of the L1 penalty creates a tradeoff. If L1 is too small, it behaves like plain linear regression, and if L1 is too large, it collapses the model by zeroing out all features. Limitations included testing with only a handful of discrete λ values. K-fold cross-validation, as seen in class, would give a more principled way to find the optimal penalty.

SVM

The most optimal test accuracy is 95.56%, achieved at $\lambda = 0.01$. The three λ values illustrate the bias-variance trade-off: $\lambda = 0.001$ overfits, $\lambda = 0.01$ balances well, and $\lambda = 0.1$ underfits. The most common misclassifications were $3 \to 8$, $9 \to 8$, and $1 \to 6$, which makes intuitive sense since those digit pairs are visually similar. The best learning rate was $\eta = 0.001$; $\eta = 0.01$ caused overshooting. Testing more than three values each of λ and η would give more confidence in the chosen hyperparameters.

MLP

Accuracy converged to 100% with a high enough number of epochs, confirming backpropagation works correctly. Larger hidden layers generally improved accuracy, without overfitting appearing. Higher learning rate ($\eta = 0.1$) performed best because the model converged faster within a fixed number of epochs. The results are limited to the model's performance for one hidden layer. The addition of more layers would allow the model to capture more complex patterns.

VI. Statement of Contributions

Tingyi Chen: Implemented and analyzed ElasticNet, Lasso, and MLP, wrote methodology and result of the models.

Raisa Chowdhury: Implemented and analyzed Multiclass SVM, wrote methodology and result of the model, discussion and conclusion.

John Wu: Conducted statistical analysis on the two datasets, wrote abstract, introduction, and datasets of the report.

References

[1] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.

[4] H. Zou and T. Hastie, "Regularization and variable selection via the elastic net," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 67, no. 2, pp. 301–320, 2005.

[5] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[7] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[8] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2022.